

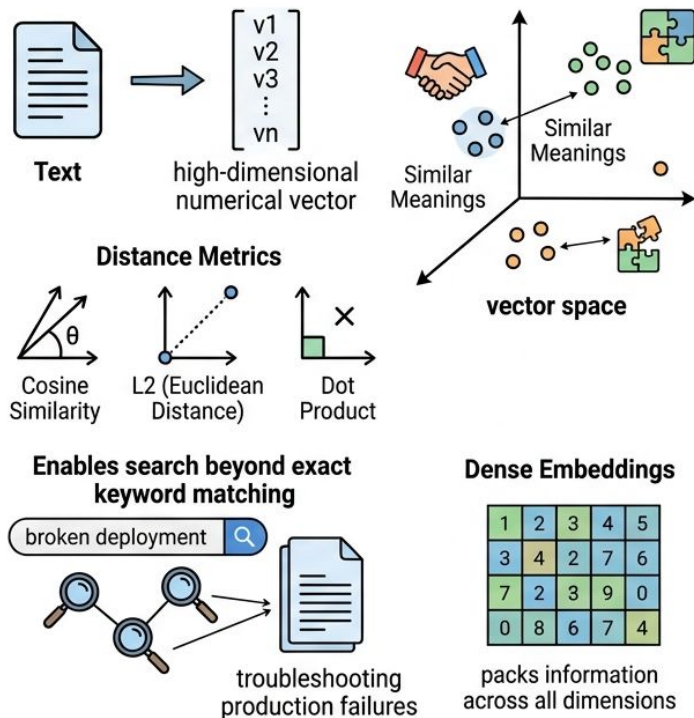


U.S. General Services
Administration

Chapter 6.1: Embeddings and RAG Fundamentals

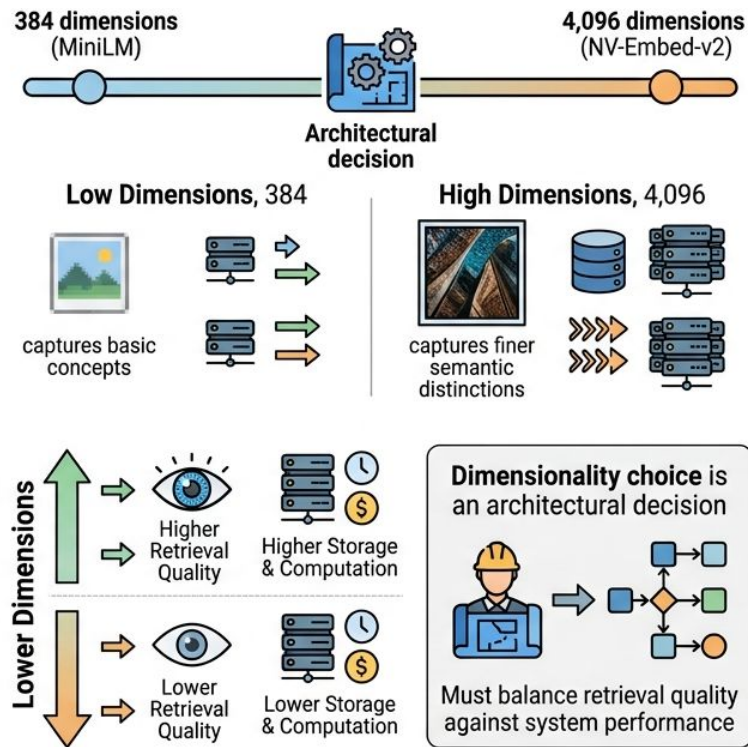
US General Service Administration
AI Community of Practice - March 2026

What Embeddings Actually Are



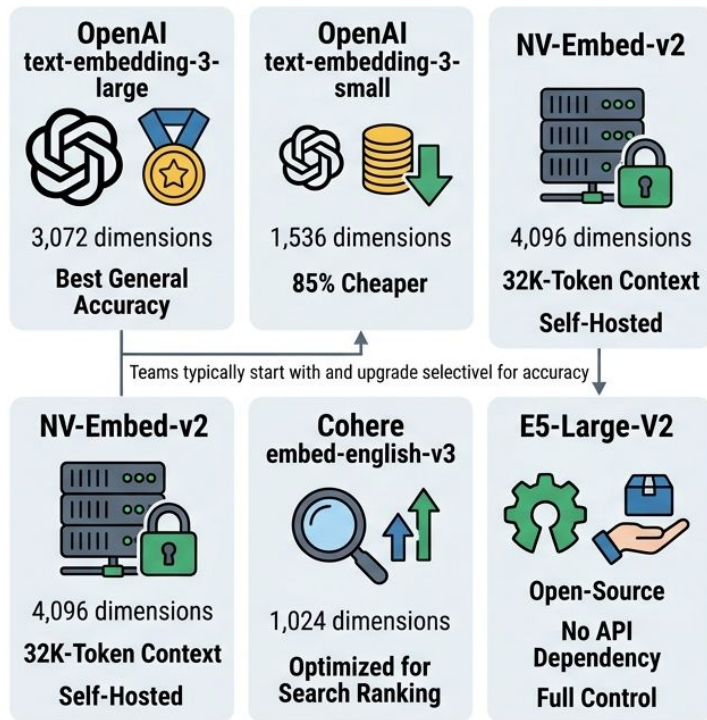
- Text becomes a high-dimensional numerical vector
- Similar meanings land near each other in vector space
- Distance metrics: cosine similarity, L2, dot product
- Enables search beyond exact keyword matching
- Dense embeddings pack information across all dimensions

Dimensionality -- The Quality vs. Cost Tradeoff



- Dimensions range from 384 (MiniLM) to 4,096 (NV-Embed-v2)
- Higher dimensions capture finer semantic distinctions
- More dimensions means more storage and computation
- Dimensionality choice is an architectural decision, not just a setting
- Must balance retrieval quality against system performance

Choosing an Embedding Model

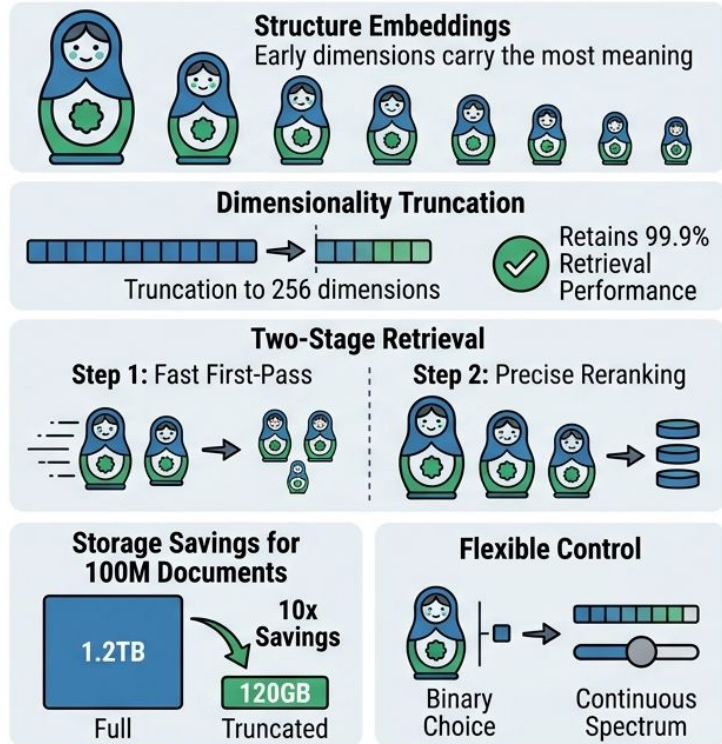


- OpenAI text-embedding-3-large: 3,072 dims, best general accuracy
- OpenAI text-embedding-3-small: 1,536 dims, 85% cheaper
- NV-Embed-v2: 4,096 dims, 32K-token context, self-hosted
- Cohere embed-english-v3: 1,024 dims, optimized for search ranking
- E5-Large-V2: open-source, no API dependency, full control

Building a Production Embedding Pipeline

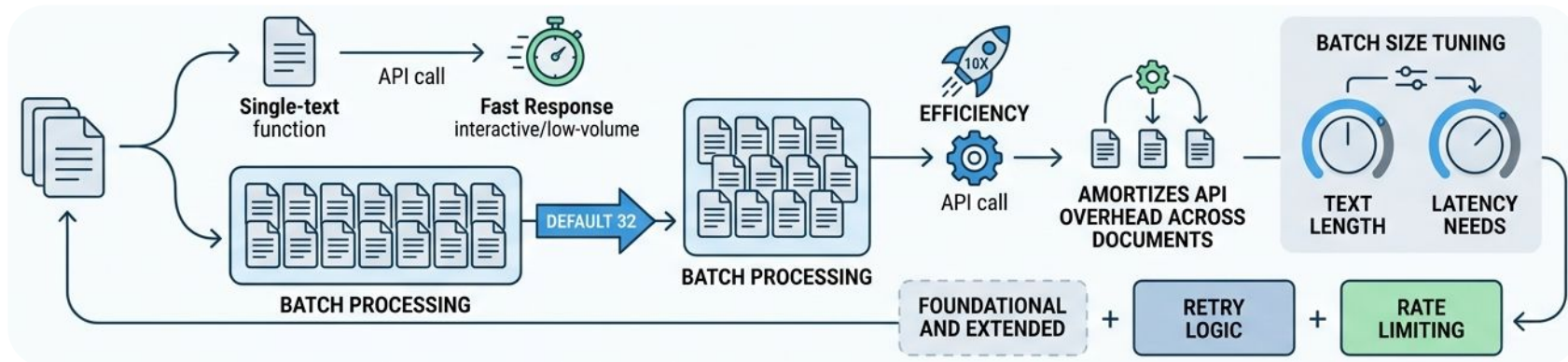
- Single-text function for interactive and low-volume use
 - Batch processing groups texts (default 32) for 10x efficiency
 - Amortizes API overhead across multiple documents
 - Batch size tuning depends on text length and latency needs
 - Foundation pattern extends with retry logic and rate limiting
- 

Matryoshka Representation Learning



- MRL structures embeddings so early dimensions carry the most meaning
- Truncation to 256 dims retains 99.9% retrieval performance
- Enables two-stage retrieval: fast first-pass, precise reranking
- 100M documents: 1.2TB full vs. 120GB truncated (10x savings)
- Transforms dimensionality from binary choice to continuous spectrum

NVIDIA NeMo Retriever and GPU Acceleration

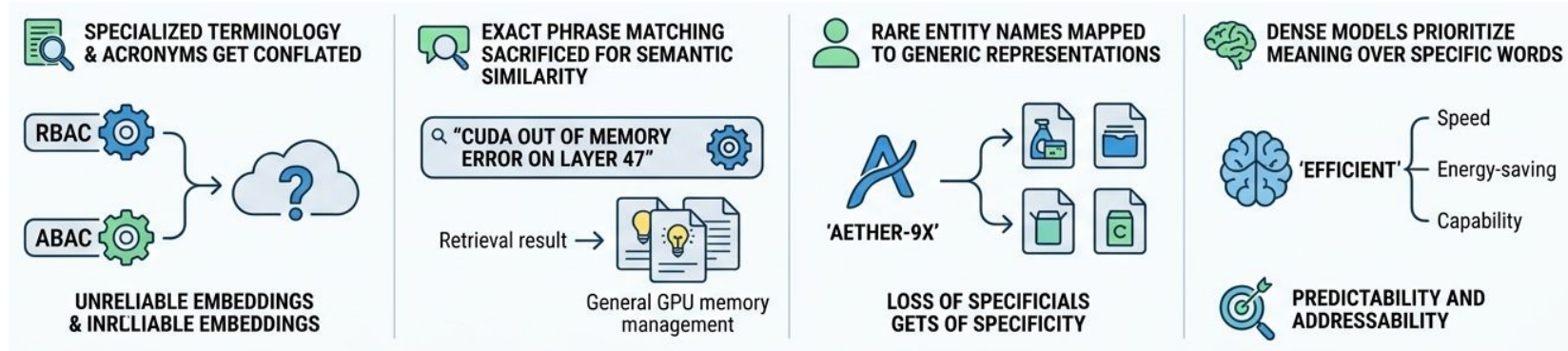


- NV-Embed-v2 via NeMo Retriever: 32K-token context, GPU-optimized
- Query latency drops from 450ms (CPU) to 35ms (GPU) -- 12.9x faster
- Batch throughput: 15 docs/sec (CPU) vs. 320 docs/sec (GPU)
- Cost: \$0.24/1K queries (CPU) vs. \$0.04/1K queries (GPU)
- Triton Inference Server handles batching and multi-GPU load balancing

Develop Anywhere, Deploy on NVIDIA

- NeMo Retriever uses an OpenAI-compatible API interface
 - Switch by changing base URL and model name only
 - Same client code, data structures, and error handling
 - Enables hybrid strategies across providers
 - Develop with OpenAI, deploy on NIM for production
- 
- A decorative graphic in the bottom right corner consisting of several overlapping blue rectangular shapes of varying shades, creating a sense of depth and movement.

Where Dense Embeddings Struggle



- Specialized terminology and acronyms get conflated
- Exact phrase matching sacrificed for semantic similarity
- Rare entity names mapped to generic representations
- Dense models prioritize meaning over specific words
- These blind spots are predictable and addressable

Hybrid Search -- Best of Both Worlds

- Combines dense vectors (semantic) with sparse vectors (keyword)
 - BM25 handles exact matching where embeddings struggle
 - Alpha parameter controls fusion weight (0.0 sparse to 1.0 dense)
 - Documents found by both methods score highest
 - Improves accuracy 15-25% over pure vector search
- 

Reciprocal Rank Fusion -- The Merging Algorithm

- RRF converts rankings into comparable scores
 - Formula: $\text{score} = \text{weight} / (\text{rank} + 60)$
 - Constant 60 prevents top results from dominating completely
 - Documents in both result sets accumulate scores from each
 - Final ranking reflects evidence from both retrieval methods
- 

Key Takeaways

- Dense embeddings power semantic search; choose models by use case
 - MRL enables flexible dimensionality for billion-scale deployments
 - GPU acceleration delivers 10x+ gains in latency, throughput, and cost
 - Hybrid search with RRF fills dense retrieval blind spots
 - Next: Chapter 6.2A -- choosing and deploying vector databases
- 